

GraphQL

IF3110 Web-Based Application Development
School of Electrical Engineering and Informatics
Institut Teknologi Bandung

Motivation

REST API still has some problems:

- **Over-fetching:** the page receives a large product representation but uses only some fields.
- **Under-fetching:** the page receives a seller ID and must make another request to learn the seller's name.

GraphQL can address both through field selection and traversal of related schema fields. REST can also offer field selection, expansion, or specialized endpoints.

What is GraphQL?

GraphQL provides a **typed schema**, a language for expressing operations, and execution rules for producing results. It has three operations:

- **Query** reads data.
- **Mutation** requests a change.
- **Subscription** receives results over time when relevant events occur.

The server implements these operations over its own databases and services. GraphQL does not require a graph database.

The schema is a public contract

The schema names available types, fields, and relationships. A client can ask for `product.seller.name` only because those fields are defined.

```
type Seller {  
  id: ID!  
  name: String!  
}  
  
type Product {  
  id: ID!  
  name: String!  
  seller: Seller  
}
```

Resolvers supply data behind the schema; clients do not choose arbitrary database columns.

Scalar, object, enum, and input types

Built-in scalars include `String`, `Int`, `Float`, `Boolean`, and `ID`.

- An **object** groups selectable output fields.
- An **enum** limits a named set of values, such as `PLACED` or `SHIPPED`.
- An **input object** groups values a client may supply to an operation.
- A custom scalar, such as `DateTime`, needs documented parsing and serialization.

Prefer a meaningful, typed field to an opaque JSON string when clients need to explore and select its parts.

Lists and nullability

GraphQL type	Meaning
<code>Product</code>	A product value or null
<code>Product!</code>	A product value is promised
<code>[Product]</code>	List or null; items may also be null
<code>[Product!]!</code>	List is present; each item is present

Query: start with the schema

```
type Query {  
  product(id: ID!) : Product  
}
```

```
type Product {  
  id: ID!  
  name: String!  
  seller: Seller!  
}
```

product accepts a required ID. It returns **Product** or **null**, for example when no accessible product matches the ID. The client can select only fields defined on **Product** and related types.

Query: operation and variables

```
query ProductPage($productId: ID!) {  
  product(id: $productId) {  
    id  
    name  
    seller { name }  
  }  
}
```

Variables:

```
{ "productId": "P101" }
```

The variable supplies the product ID without changing the operation text. The selection names the exact result fields requested by this client.

Query: response

```
{
  "data": {
    "product": {
      "id": "P101",
      "name": "Travel Mug",
      "seller": { "name": "Maya's Shop" }
    }
  }
}
```

The JSON structure under `data` follows the operation's selection. If no accessible product matches, `data.product` can be `null`. Selected fields can still require several backend lookups.

Mutation: design the schema

```
input PlaceOrderInput {  
  cartId: ID!  
}  
  
type PlaceOrderPayload {  
  order: Order  
  userErrors: [CheckoutError!]!  
}  
  
type Order { id: ID! status: String! }  
type CheckoutError { code: String! message: String! }  
  
type Mutation {  
  placeOrder(input: PlaceOrderInput!): PlaceOrderPayload!  
}
```

The input contains a cart ID, while the server obtains customer identity from authentication and calculates prices itself. The payload carries a result the client can inspect.

Mutation: operation and variables

```
mutation Checkout($input: PlaceOrderInput!) {  
  placeOrder(input: $input) {  
    order { id status }  
    userErrors { code message }  
  }  
}
```

Variables:

```
{ "input": { "cartId": "K12" } }
```

The client asks for the order fields it needs after checkout. The server still checks cart ownership, stock, and the final amount.

Mutation: success and business failure

Successful checkout:

```
{ "data": { "placeOrder": { "order": { "id": "055", "status": "PLACED" }, "userErrors": [] } } }
```

Out-of-stock outcome:

```
{ "data": { "placeOrder": { "order": null, "userErrors": [ { "code": "OUT_OF_STOCK", "message": "Travel Mug is unavailable" } ] } } }
```

Document the result invariant. A lost response may cause the client to retry, so checkout also needs a server-side strategy to prevent duplicate orders.

Subscription: order updates

Schema:

```
type Subscription {  
  orderStatusChanged(orderId: ID!) : OrderStatusEvent!  
}  
  
type OrderStatusEvent { status: String! occurredAt: String! }
```

Operation:

```
subscription TrackOrder($id: ID!) {  
  orderStatusChanged(orderId: $id) { status occurredAt }  
}
```

When the order changes, the client may receive a result such as

```
{ "data": { "orderStatusChanged": { "status": "SHIPPED", "occurredAt": "2026-09-26T09:15:00Z" } } }.
```

The server needs an event source and streaming transport. Check order access for each subscriber. After reconnecting, fetch the current status because events may have been missed.

Resolver performance: the N+1 problem

A query returns 20 products. If each `Product.seller` resolver performs another lookup, the backend may make **1 product lookup + 20 seller lookups**.

Batch seller IDs within the request and map the results back to their products. This may reduce the example to two lookups.

One GraphQL network request can still do substantial backend work. Measure data-source calls and resolver time. Keep request-scoped caches isolated between users.

Authorization and query cost

- Enforce order ownership in shared business logic. An order may be reachable through `Query.order`, `Customer.orders`, or a subscription.
- Validate inputs and enforce maximum collection sizes, query depth or estimated cost, and execution deadlines.
- Monitor expensive fields. A request-count rate limit alone does not bound the work in one GraphQL request.

The schema verifies types and shape. It does not grant permission to access a field.

Errors, documentation, and change

- Use a consistent typed result for expected checkout failures. Unexpected resolver failures can appear in the GraphQL `errors` array, sometimes alongside partial `data`.
- Describe types, fields, arguments, inputs, pagination, and authorization. Supply example operations and responses.
- Add new fields compatibly when possible. Deprecate old fields with a replacement and migration guidance before removing them.

```
sellerName: String @deprecated(reason: "Use  
seller { name }")
```

Introspection helps clients discover the schema; it does not explain every business rule.

GraphQL practices

Prefer	Avoid
A small typed domain schema	Opaque JSON strings and exposed tables
Bounded collections	An unbounded <code>products</code> field
Batched nested data access	One seller lookup per product
Authorization on every data path	A check on just one root query
Documented evolution	Removing a used field without notice